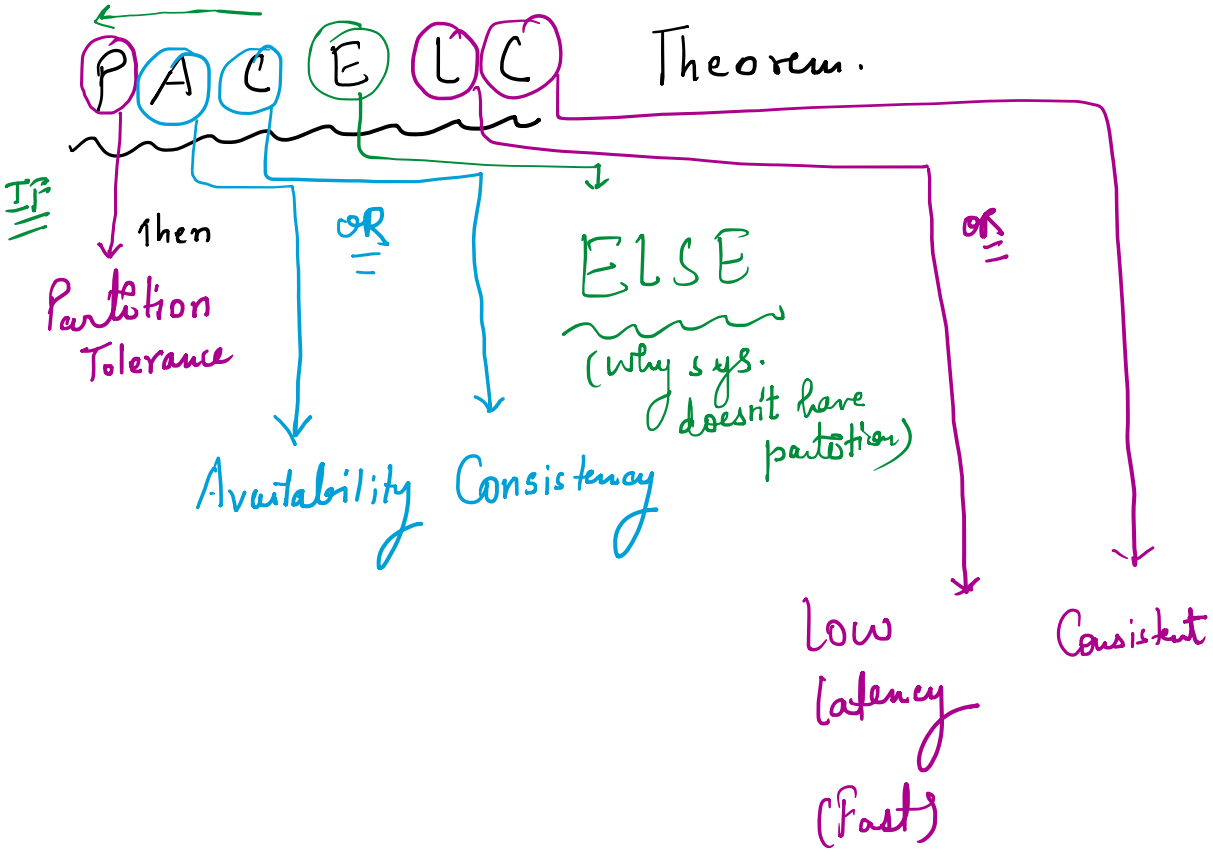
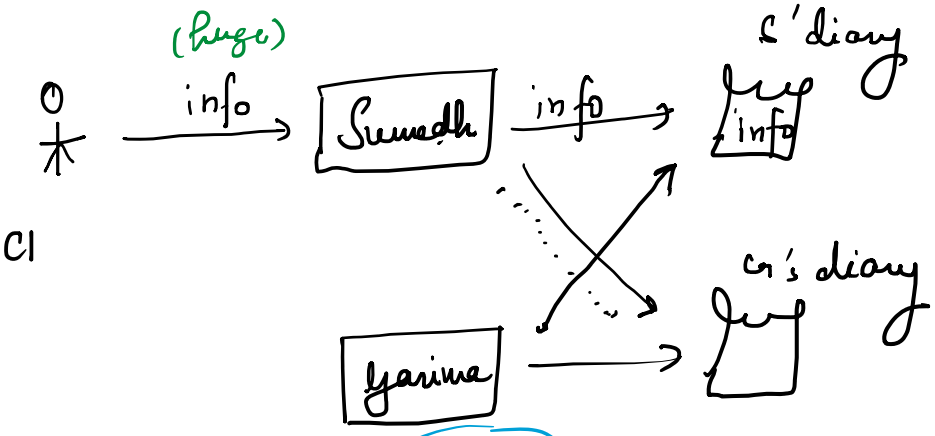
Wednesday, October 7, 2026 8:44 PM



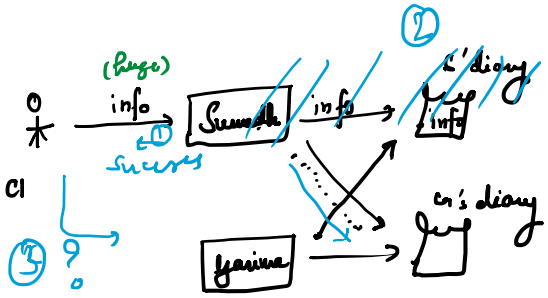


2 options

Since the 'info' itself is huge, just send an instantaneous 'Success' to Cl & thereafter asynn write to G's diary

Ensure the 'info' (even if huge) is completely synced to G's diary, then only send Success to ex.

& thereafter async write to G's diary



Fast ✓
(Low Latency)

Consistency
X

Slow X
(High Latency)

Consistency
✓

9's of availability → 99%
99.9%
99.99%

(*) Master-Slave Archi :-

(Leader-Follower Archi.)

Data is stored in DB → multiple [Sharding].

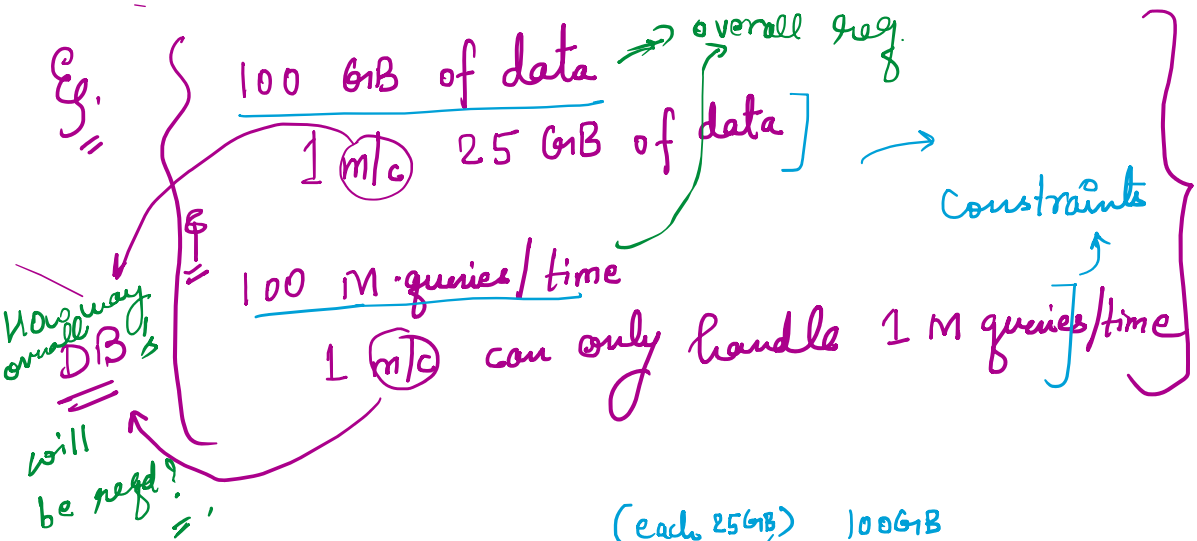
Assume, data can fit in 1 m/c :-

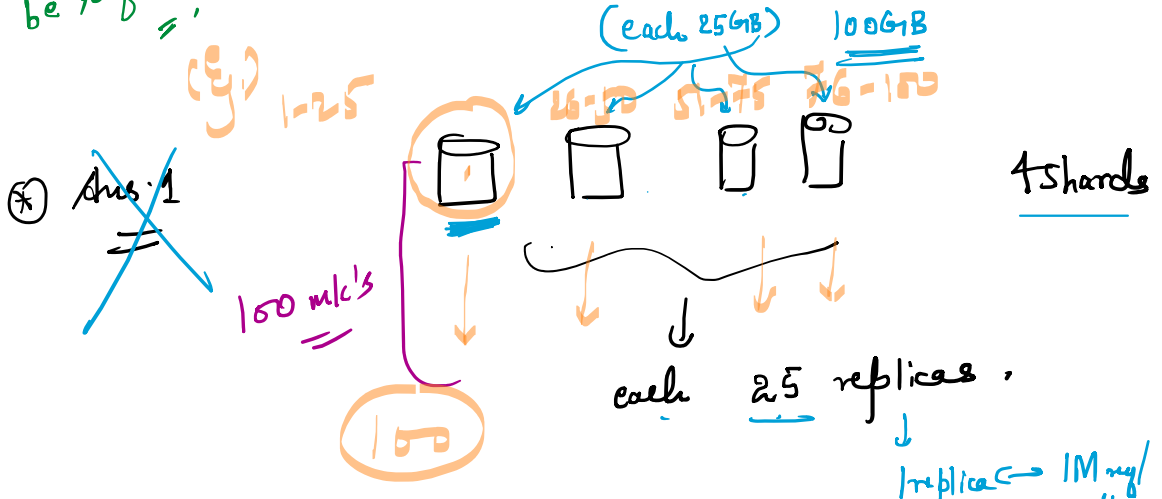
Backup (1.) SPOF :-

Another (2.) Bottleneck :- A lot of queries coming to the same m/c.
m/c's to share load wl

Sharding – When the size of data can't fit in 1 m/c.

Replica – # of queries can't be handled by 1 m/c.





(*) ~~Ans 1~~

(*) ~~Ans 2~~ 100 m/c's

⊛ ~~Ans: 2~~

100 m/c's

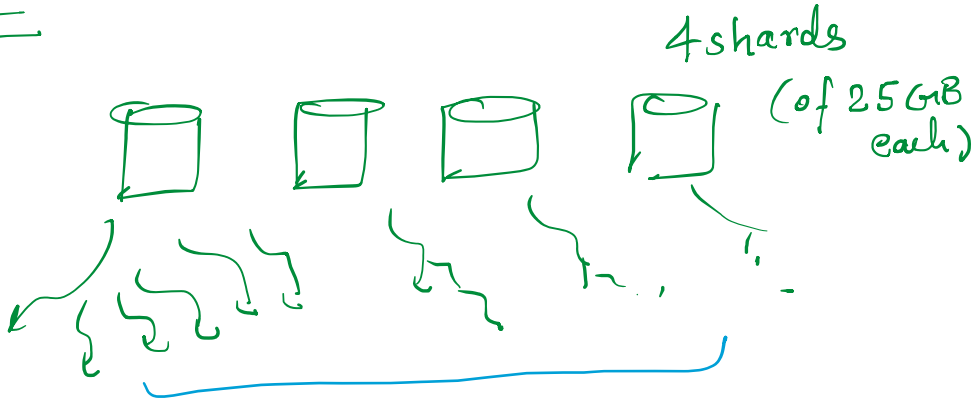
(no replica)

16GB data per m/c ~

↓

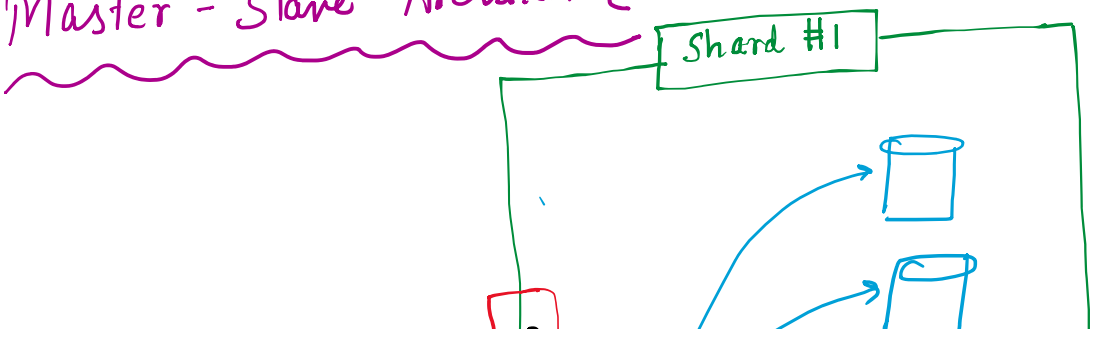
each such m/c (& hence data) can have 100 M queries/time ?

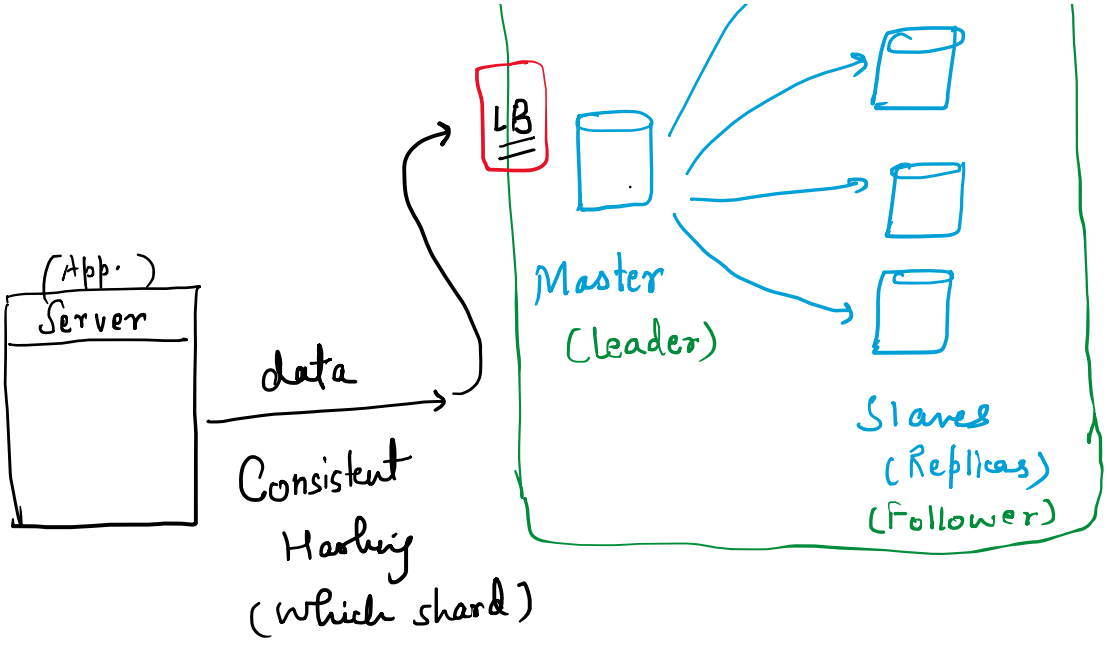
SOL^n



Each shard has 100 replicas.

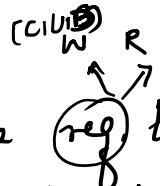
Master - Slave Architecture





Which replica of shard ?

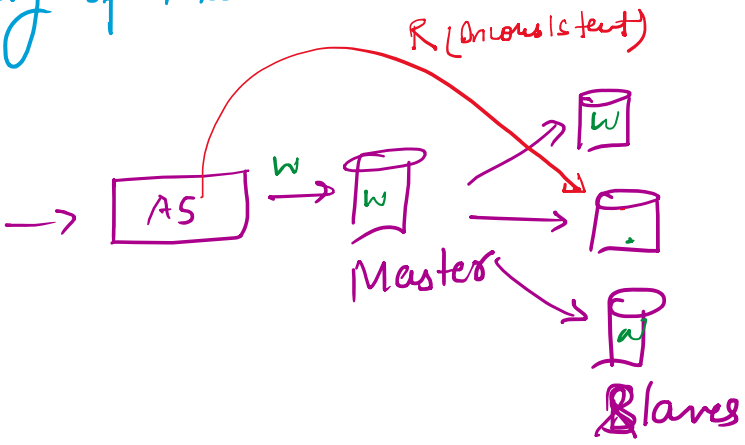
AS should send the req. to 1 m/c & that m/c should distribute to others.



M/c that receives 'Write' req. → Master

" " " 'Read' " → Slaves.

Duty of Master to send it to others (slaves)

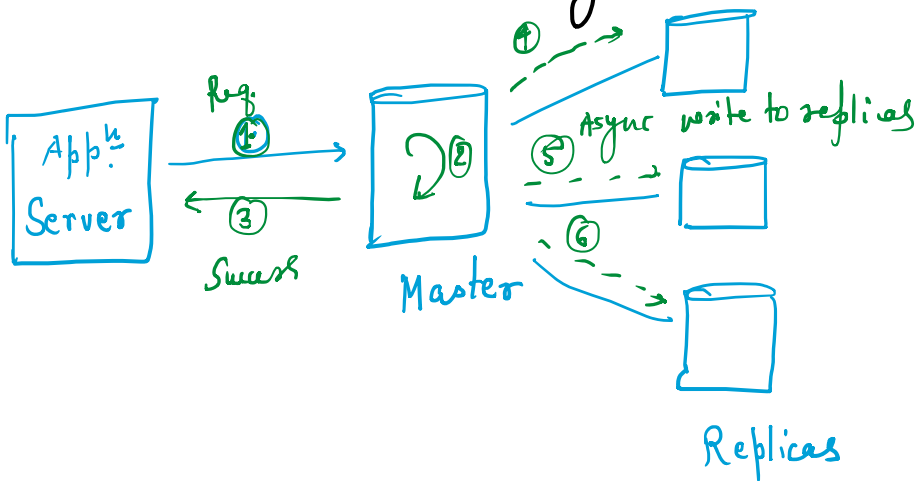


(*) When will master write to slaves?

↳ do we need consistency or availability.

1. No req. of consistency.

(Even if sys. is never consistent, we're okay)



Possible that just after (3.), master goes down (dies)

=) even 1 replica won't have the write

=) data lost forever

=) never consistent.

Eg: YT Likes, Views

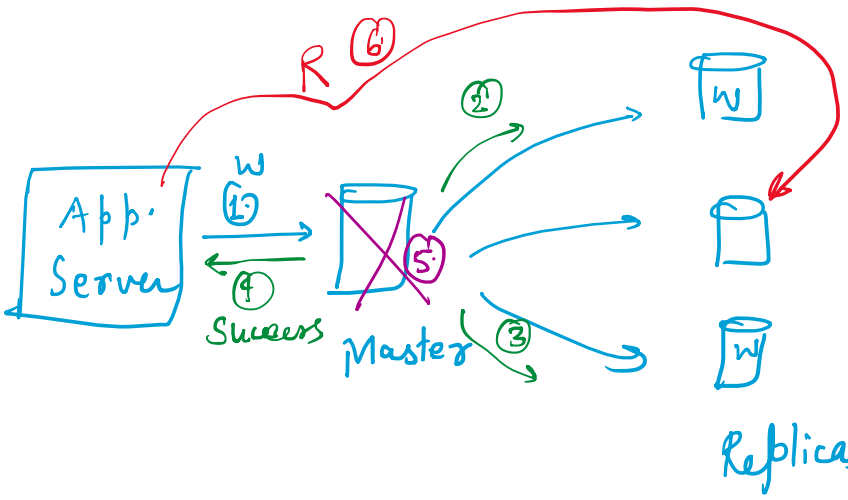
Availability ↑↑ Consistency ↓↓

2. No req of consistency BUT

② No req. of consistency BUT ☹ though want the sys. to be eventually consistent.

⇒ Data loss isn't acceptable

Eg. FB News Feed.



Master sends the 'W' req. to all replicas, & the moment a particular % Threshold responds back, we return success to AS.

NOW,

If master dies, data will be available w/ at least few replicas

Note

When master dies, one of replicas become master.

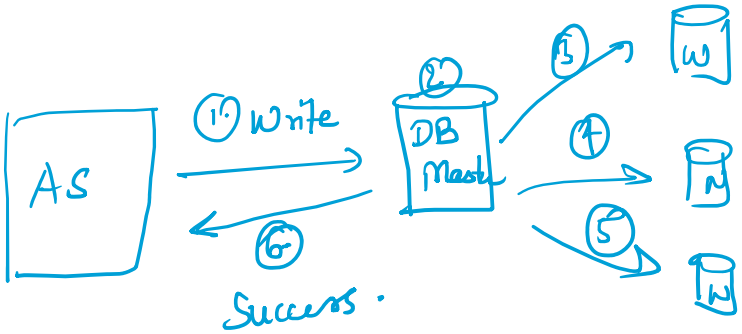
(Leader Election Algo)

'Eventual Consistency'

(3) Consistency is MUST

Eg. → Banking Appln

Writes to Master is consider successful only (& only if) ALL the replicas receive the write



Consistency ↑ ↓ Availability

Replicas

Write with Read

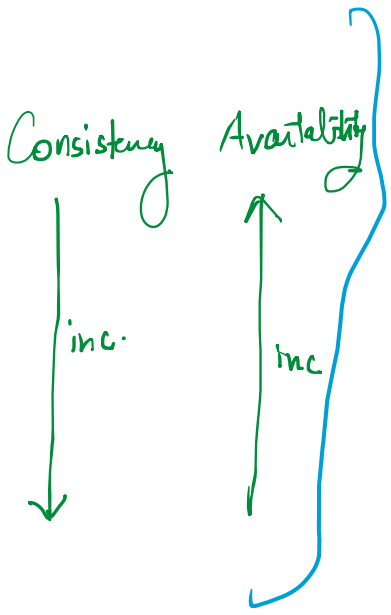
Consistent.

Note → Eventual - Consistency ≠ Consistency

Note → Writes are sent to Master
Read req. are sent to any of the server @ random.

⊛ Summary

1. Inconsistent
2. Eventual-consistent
3. Consistent



CAP Theorem in disguise

⊛ Pros of Master-Slave ?

- ↳ Backup (SPOF avoided)
- ↳ Replicas distribute Load (Bottleneck ↓)

⊛ Cons

1. We decide #replicas basis #reads-sec

1.) We decide # replicas basis # reads req

But if I want consistent sys.

⇒ Writes become v v slow

⇒ Latency ↑

2.) Huge # of writes [write-heavy sys.]

↳ master becomes bottleneck

↓

Sol^{n} : Multi-Master setup